

PYTHON & REST API INTEGRATION

Flask · REST APIs · JSON/XML · Authentication · Postman · Data Integration

Comprehensive Instructor-Led Teaching Guide

Trainer: Prachi Kabra

Duration	8 Hours (2 Days)
Level	Intermediate
Prerequisites	Python basics, HTTP fundamentals

1. Course Overview & Learning Objectives

1.1 Course Description

This module equips participants with the skills to build and consume REST APIs using Python (Flask), handle data serialisation formats (JSON and XML), implement standard authentication strategies, and test integrations using Postman. By the end, participants will be able to design end-to-end data integration pipelines that connect disparate systems.

1.2 Learning Objectives

Upon completion, participants will be able to:

- Understand the HTTP request/response lifecycle and REST architectural principles
- Build REST APIs from scratch using the Flask micro-framework in Python
- Parse, produce, and transform JSON and XML payloads programmatically
- Apply API Key, HTTP Basic, and JWT authentication to secure endpoints
- Use Postman to design, test, and document API collections
- Architect a data integration solution that pulls data from multiple API sources

1.3 Agenda at a Glance

Module	Topic	Duration
1	Flask & REST Fundamentals	2 hrs
2	JSON & XML Handling	1.5 hrs
3	Authentication Strategies	1.5 hrs
4	Postman Hands-On	1.5 hrs
5	Data Integration Capstone	1.5 hrs

2. Module 1 — Flask & REST API Fundamentals

2.1 What is an API?

An API (Application Programming Interface) is a contract that defines how software components communicate. A REST (Representational State Transfer) API uses HTTP as its transport and follows stateless, resource-oriented principles defined by Roy Fielding.

Key REST Constraints

- Client–Server: separation of concerns between UI and data storage
- Stateless: each request carries all information needed; no session state on server
- Cacheable: responses must declare whether they are cacheable
- Uniform Interface: resources identified by URIs; CRUD mapped to HTTP verbs
- Layered System: client cannot tell whether it is directly connected to the server

2.2 HTTP Methods & Status Codes

Method	Purpose	Typical Status
GET	Retrieve a resource	200 OK
POST	Create a new resource	201 Created
PUT	Replace entire resource	200 / 204
PATCH	Partial update	200 / 204
DELETE	Remove resource	204 No Content

2.3 Introducing Flask

Flask is a lightweight WSGI web framework in Python. Its minimalist core makes it ideal for building REST APIs — you get routing, request parsing, and response building with almost no boilerplate.

Installation

```
pip install flask flask-restful
```

Your First Flask API (Hello World)

```
from flask import Flask, jsonify

app = Flask(__name__)
```

```
@app.route('/api/hello', methods=['GET'])
def hello():
    return jsonify({'message': 'Hello, World!', 'status': 'success'}), 200

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

2.4 Building a Full CRUD API

We will build a products API that supports Create, Read, Update, and Delete operations on an in-memory data store.

Project Structure

```
flask_api/
├── app.py          # Main application
├── models.py       # Data models
├── routes/
│   ├── products.py # Product endpoints
│   └── auth.py     # Auth endpoints
└── utils.py       # Helpers
```

CRUD Endpoints — products.py

```
from flask import Blueprint, jsonify, request, abort

products_bp = Blueprint('products', __name__, url_prefix='/api/v1')

# In-memory store
products = {}
next_id = 1

# GET all products
@products_bp.route('/products', methods=['GET'])
def get_products():
    return jsonify({'data': list(products.values()), 'count': len(products)}), 200

# GET single product
@products_bp.route('/products/<int:product_id>', methods=['GET'])
def get_product(product_id):
    product = products.get(product_id)
    if not product:
        abort(404, description=f'Product {product_id} not found')
    return jsonify({'data': product}), 200

# POST create product
@products_bp.route('/products', methods=['POST'])
def create_product():
```

```

    global next_id
    body = request.get_json(force=True)
    if not body or 'name' not in body:
        abort(400, description='Field "name" is required')
    product = {'id': next_id, 'name': body['name'], 'price': body.get('price', 0.0)}
    products[next_id] = product
    next_id += 1
    return jsonify({'data': product}), 201

# PUT update product
@products_bp.route('/products/<int:product_id>', methods=['PUT'])
def update_product(product_id):
    if product_id not in products:
        abort(404)
    body = request.get_json(force=True)
    products[product_id].update({'name': body.get('name'), 'price':
body.get('price')})
    return jsonify({'data': products[product_id]}), 200

# DELETE product
@products_bp.route('/products/<int:product_id>', methods=['DELETE'])
def delete_product(product_id):
    if product_id not in products:
        abort(404)
    del products[product_id]
    return '', 204

```

2.5 Error Handling & Response Standards

Consistent error responses improve the developer experience for API consumers. Flask's `errorhandler` decorator centralises error formatting.

```

from flask import jsonify

@app.errorhandler(400)
def bad_request(e):
    return jsonify({'error': 'Bad Request', 'message': str(e)}), 400

@app.errorhandler(404)
def not_found(e):
    return jsonify({'error': 'Not Found', 'message': str(e)}), 404

@app.errorhandler(500)
def internal(e):
    return jsonify({'error': 'Internal Server Error'}), 500

```



TRAINER TIP — Whiteboard Exercise

Draw the request lifecycle on the board: Client → HTTP Request → Flask Router → View Function → Response.

Ask learners: 'What happens if the view function raises an unhandled exception?'

3. Module 2 — JSON & XML Data Serialisation

3.1 JSON (JavaScript Object Notation)

JSON is the de-facto standard payload format for REST APIs. Python's built-in `json` module (and Flask's `jsonify`) make serialisation trivial.

Core Operations

```
import json

# Python dict → JSON string
data = {'user': 'Prachi', 'score': 98, 'active': True}
json_str = json.dumps(data, indent=2)
print(json_str)

# JSON string → Python dict
parsed = json.loads(json_str)
print(parsed['user'])  # 'Prachi'

# Read JSON file
with open('config.json') as f:
    config = json.load(f)

# Write JSON file
with open('output.json', 'w') as f:
    json.dump(data, f, indent=2)
```

Working with Nested JSON

```
response = {
    'status': 'success',
    'data': {
        'orders': [
            {'id': 1, 'items': [{'sku': 'A1', 'qty': 3}]},
            {'id': 2, 'items': [{'sku': 'B2', 'qty': 1}]},
        ]
    }
}

# Safe deep access
orders = response.get('data', {}).get('orders', [])
for order in orders:
    for item in order.get('items', []):
        print(f"Order {order['id']} - SKU: {item['sku']}, Qty: {item['qty']}")
```

3.2 XML (eXtensible Markup Language)

Many legacy enterprise systems and SOAP-based APIs still use XML. Python provides `xml.etree.ElementTree` in the standard library and the more powerful `lxml` library for advanced needs.

Parsing XML

```
import xml.etree.ElementTree as ET

xml_data = '''
<catalog>
  <product id='101'>
    <name>Laptop</name>
    <price currency='USD'>1299.99</price>
  </product>
  <product id='102'>
    <name>Mouse</name>
    <price currency='USD'>25.00</price>
  </product>
</catalog>
'''

root = ET.fromstring(xml_data)
for product in root.findall('product'):
    pid    = product.get('id')
    name   = product.find('name').text
    price  = product.find('price').text
    currency = product.find('price').get('currency')
    print(f"{pid} | {name} | {currency} {price}")
```

Building XML Programmatically

```
import xml.etree.ElementTree as ET

root = ET.Element('response')
root.set('version', '1.0')

status = ET.SubElement(root, 'status')
status.text = 'OK'

item = ET.SubElement(root, 'item')
item.set('id', '1')
ET.SubElement(item, 'name').text = 'Keyboard'
ET.SubElement(item, 'price').text = '89.99'

tree = ET.ElementTree(root)
ET.indent(tree, space=' ') # pretty-print (Python 3.9+)
tree.write('catalog.xml', encoding='unicode', xml_declaration=True)
```


JSON ↔ XML Conversion

```
import json, xmltodict # pip install xmltodict

# XML to JSON
xml_str = '<person><name>Prachi</name><age>30</age></person>'
as_dict = xmltodict.parse(xml_str)
as_json = json.dumps(as_dict, indent=2)
print(as_json)

# JSON to XML
data = {'person': {'name': 'Prachi', 'age': 30}}
as_xml = xmltodict.unparse(data, pretty=True)
print(as_xml)
```

3.3 Serving XML from Flask

```
from flask import Flask, Response
import xml.etree.ElementTree as ET

app = Flask(__name__)

@app.route('/api/products/xml')
def products_xml():
    root = ET.Element('products')
    for pid, name in [(1, 'Laptop'), (2, 'Mouse')]:
        p = ET.SubElement(root, 'product', id=str(pid))
        ET.SubElement(p, 'name').text = name
    xml_str = ET.tostring(root, encoding='unicode')
    return Response(xml_str, mimetype='application/xml')
```

TRAINER NOTE — Content Negotiation

REST APIs can serve both JSON and XML from the same endpoint using the Accept request header.

Demonstrate: `curl -H 'Accept: application/xml' http://localhost:5000/api/products`

Production APIs use the Accept header to switch serialisation format dynamically.

4. Module 3 — API Authentication & Security

4.1 Why Authentication Matters

Authentication verifies the identity of the caller. Without it, any client can read or modify your data. Three strategies are covered: API Keys, HTTP Basic Auth, and JWT (JSON Web Tokens).

4.2 API Key Authentication

API keys are opaque tokens passed in a request header. They are simple to implement but do not carry claims — the server must look up the key on every request.

```
from flask import Flask, request, jsonify, abort
from functools import wraps

app = Flask(__name__)

VALID_KEYS = {'key-prod-001', 'key-dev-999'} # In production: use a database

def require_api_key(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        key = request.headers.get('X-API-Key')
        if not key or key not in VALID_KEYS:
            abort(401, description='Invalid or missing API key')
        return f(*args, **kwargs)
    return decorated

@app.route('/api/secure/data')
@require_api_key
def secure_data():
    return jsonify({'secret': 'Here is protected data'}), 200
```

4.3 HTTP Basic Authentication

HTTP Basic Auth encodes credentials (username:password) in Base64 and sends them in the Authorization header. It MUST be used over HTTPS to avoid credential exposure.

```
import base64
from flask import request, abort
from functools import wraps

USERS = {'admin': 'secret123', 'prachi': 'pass456'}

def basic_auth(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        auth_header = request.headers.get('Authorization', '')
```

```

    if not auth_header.startswith('Basic '):
        abort(401)
    try:
        encoded = auth_header.split(' ')[1]
        decoded = base64.b64decode(encoded).decode('utf-8')
        username, password = decoded.split(':', 1)
    except Exception:
        abort(401)
    if USERS.get(username) != password:
        abort(401, description='Invalid credentials')
    return f(*args, **kwargs)
return decorated

```

4.4 JWT (JSON Web Tokens)

JWTs are self-contained tokens that carry cryptographically signed claims. The server issues a token at login; subsequent requests present the token — no server-side session storage required.

JWT Structure

Part	Content	Example
Header	Algorithm & token type	{"alg":"HS256","typ":"JWT"}
Payload	Claims (sub, exp, roles)	{"sub":"user1","exp":1710000000}
Signature	HMAC of header + payload	Base64Url(HMAC-SHA256(h+p, secret))

Implementing JWT with PyJWT

```

import jwt, datetime
from flask import Flask, request, jsonify, abort
from functools import wraps

app = Flask(__name__)
SECRET = 'super-secret-key-change-in-prod'

def generate_token(user_id: str) -> str:
    payload = {
        'sub': user_id,
        'iat': datetime.datetime.utcnow(),
        'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
    }
    return jwt.encode(payload, SECRET, algorithm='HS256')

def jwt_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        auth = request.headers.get('Authorization', '')

```

```

    if not auth.startswith('Bearer '):
        abort(401, description='Missing Bearer token')
    token = auth.split(' ')[1]
    try:
        payload = jwt.decode(token, SECRET, algorithms=['HS256'])
        request.user_id = payload['sub']
    except jwt.ExpiredSignatureError:
        abort(401, description='Token has expired')
    except jwt.InvalidTokenError:
        abort(401, description='Invalid token')
    return f(*args, **kwargs)
return decorated

@app.route('/auth/login', methods=['POST'])
def login():
    body = request.get_json()
    # In production: validate against DB
    if body.get('username') == 'admin' and body.get('password') == 'secret':
        token = generate_token('admin')
        return jsonify({'token': token}), 200
    abort(401, description='Wrong credentials')

@app.route('/api/protected')
@jwt_required
def protected():
    return jsonify({'message': f'Hello, {request.user_id}!'}), 200

```

4.5 Comparison Matrix

Criterion	API Key	Basic Auth	JWT
Stateless	✓ Yes	✓ Yes	✓ Yes
Carries Claims	✗ No	✗ No	✓ Yes
Easy to Revoke	✓ Yes	✓ Yes	⚠ Complex
Best For	M2M	Internal	User-facing

5. Module 4 — Postman Hands-On

5.1 Postman Overview

Postman is the industry-standard API client for designing, testing, and documenting APIs. Key features: Collections, Environments, Pre-request Scripts, Test Scripts, and Newman (CLI runner).

5.2 Setting Up a Collection

1. Open Postman → New Collection → Name it 'Flask Products API'
2. Create an Environment: 'Dev' with variable `BASE_URL` = `http://localhost:5000`
3. Use `{{BASE_URL}}` in all request URLs

5.3 Request Types — Step-by-Step

POST — Create a Product

```
Method : POST
URL      : {{BASE_URL}}/api/v1/products
Headers: Content-Type: application/json
Body (raw JSON):
{
  "name": "Wireless Keyboard",
  "price": 45.99
}
```

GET — List All Products

```
Method : GET
URL      : {{BASE_URL}}/api/v1/products
Headers: X-API-Key: key-dev-999    (if API key auth enabled)
```

PUT — Update a Product

```
Method : PUT
URL      : {{BASE_URL}}/api/v1/products/1
Body (raw JSON):
{
  "name": "Mechanical Keyboard",
  "price": 89.99
}
```

DELETE — Remove a Product

```
Method : DELETE
URL      : {{BASE_URL}}/api/v1/products/1
```

```
Expected status: 204 No Content
```

5.4 Writing Postman Tests

Postman's Tests tab executes JavaScript after the response is received. Assertions use the `pm.expect` Chai-style API.

```
// Test: status code is 201
pm.test('Status is 201', () => {
  pm.response.to.have.status(201);
});

// Test: response has 'data' key
pm.test('Response has data', () => {
  const body = pm.response.json();
  pm.expect(body).to.have.property('data');
});

// Save product ID for next request
const id = pm.response.json().data.id;
pm.environment.set('product_id', id);
```

5.5 JWT Auth Flow in Postman

4. Send POST `/auth/login` with credentials in body → copy the returned token
5. In Collection → Authorization tab → select Bearer Token
6. Paste token value; all requests in the collection will inherit it
7. Use Pre-request Script to auto-refresh token before expiry

```
// Pre-request Script - auto-login if token missing
const token = pm.environment.get('jwt_token');
if (!token) {
  pm.sendRequest({
    url: pm.environment.get('BASE_URL') + '/auth/login',
    method: 'POST',
    header: { 'Content-Type': 'application/json' },
    body: { mode: 'raw', raw: JSON.stringify({username:'admin',password:'secret'}) }
  }, (err, res) => {
    pm.environment.set('jwt_token', res.json().token);
  });
}
```

5.6 Running Collections with Newman

Newman is Postman's CLI runner — ideal for CI/CD pipelines.

```
npm install -g newman
```

```
# Export collection as flask_api.postman_collection.json
newman run flask_api.postman_collection.json \
  --environment dev.postman_environment.json \
  --reporters cli,json \
  --reporter-json-export results.json
```

6. Module 5 — Data Integration Across Systems

6.1 What is Data Integration?

Data integration combines data from multiple source systems into a unified view. REST APIs are the primary integration fabric in modern data architectures — they expose data from databases, SaaS platforms, IoT devices, and microservices.

6.2 Consuming External APIs with Python requests

```
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

# Resilient session with retry + timeout
def build_session(retries=3, backoff=0.5) -> requests.Session:
    session = requests.Session()
    retry = Retry(
        total=retries,
        backoff_factor=backoff,
        status_forcelist=[429, 500, 502, 503, 504]
    )
    adapter = HTTPAdapter(max_retries=retry)
    session.mount('http://', adapter)
    session.mount('https://', adapter)
    return session

session = build_session()

# GET with auth header
resp = session.get(
    'https://api.example.com/v2/orders',
    headers={'Authorization': 'Bearer <your-token>'},
    params={'status': 'shipped', 'limit': 100},
    timeout=10
)
resp.raise_for_status() # Raises HTTPError on 4xx/5xx
orders = resp.json()
```

6.3 Pagination Handling

Most production APIs paginate large datasets. Two common patterns: cursor-based and offset-based.

```
def fetch_all_pages(session, base_url, headers, page_size=100):
    """Offset-based pagination - fetches all records"""
    results = []
    offset = 0
```



```
while True:
    resp = session.get(
        base_url,
        headers=headers,
        params={'limit': page_size, 'offset': offset},
        timeout=10
    )
    resp.raise_for_status()
    data = resp.json()
    page_records = data.get('data', [])
    results.extend(page_records)
    if len(page_records) < page_size:
        break # last page
    offset += page_size
return results
```

6.4 Capstone — Multi-API Integration Pipeline

Scenario: Pull product data from your Flask API, enrich it with currency conversion from a public FX API, and save the result as a JSON file ready for downstream consumption.

```
import requests, json
from datetime import datetime

BASE      = 'http://localhost:5000/api/v1'
FX_URL    = 'https://open.er-api.com/v6/latest/USD'
API_KEY   = 'key-dev-999'

def get_products():
    r = requests.get(f'{BASE}/products',
                    headers={'X-API-Key': API_KEY}, timeout=5)
    r.raise_for_status()
    return r.json()['data']

def get_fx_rates(base='USD'):
    r = requests.get(FX_URL, timeout=5)
    r.raise_for_status()
    return r.json().get('rates', {})

def enrich_products(products, rates, target='INR'):
    rate = rates.get(target, 1)
    enriched = []
    for p in products:
        enriched.append({
            **p,
            f'price_{target}': round(p['price'] * rate, 2),
            'fx_rate': rate,
            'enriched_at': datetime.utcnow().isoformat()
        })
    return enriched
```

```
if __name__ == '__main__':
    products = get_products()
    rates     = get_fx_rates()
    enriched = enrich_products(products, rates)
    with open('enriched_products.json', 'w') as f:
        json.dump({'data': enriched, 'count': len(enriched)}, f, indent=2)
    print(f'Saved {len(enriched)} enriched products.')
```

6.5 Error Handling Best Practices

Scenario	Exception	Handling Strategy
Network timeout	requests.Timeout	Retry with exponential backoff
4xx client error	HTTPError	Log and skip or raise
5xx server error	HTTPError	Retry up to N times
JSON parse fail	ValueError	Fall back to raw text, log
Missing key	KeyError	Use .get() with defaults

7. Assessment & Knowledge Check

7.1 Quiz Questions

8. What HTTP method should be used to partially update a resource? (a) POST (b) PUT (c) PATCH (d) GET
9. What is the correct HTTP status code for a successful resource creation?
10. Which authentication type stores user identity claims inside the token itself?
11. In Flask, how do you access a JSON request body? Write the one-line code.
12. What is the purpose of the Content-Type: application/json header?

7.2 Practical Assignment

Build a Flask REST API for a student records system with the following requirements:

- Endpoints: GET /students, GET /students/<id>, POST /students, PUT /students/<id>, DELETE /students/<id>
- Fields: id, name, email, grade, enrolled_date
- Secure all endpoints with JWT authentication
- Return XML responses when the client sends Accept: application/xml
- Add a GET /students/export endpoint that returns a JSON file download
- Test all endpoints in Postman with at least 2 assertions per request

7.3 Evaluation Rubric

Criterion	Max Marks	Weight
All CRUD endpoints functional	20	20%
JWT authentication implemented	20	20%
JSON & XML both supported	20	20%
Postman collection with tests	20	20%
Code quality & error handling	20	20%

8. Quick Reference & Cheat Sheet

Flask Decorators

Decorator	Purpose
@app.route('/path', methods=['GET'])	Define a GET endpoint
@app.errorhandler(404)	Custom 404 handler
@app.before_request	Run before every request
@app.after_request	Run after every response

requests Library Cheat Sheet

```
import requests
r = requests.get(url, params={}, headers={}, timeout=10)
r = requests.post(url, json={}, headers={})
r = requests.put(url, json={}, headers={})
r = requests.delete(url, headers={})
r.status_code    # HTTP status
r.json()         # Parse JSON body
r.text           # Raw string body
r.raise_for_status() # Raise on 4xx/5xx
```

Useful pip Packages

Package	Purpose
flask	Core web framework
flask-restful	REST resource abstraction
PyJWT	JWT encoding / decoding
requests	HTTP client
xmltodict	XML ↔ dict conversion
marshmallow	Schema validation & serialisation
python-dotenv	Load .env configuration files

Trainer: Prachi Kabra | Document Version 1.0